

SurSadhana AI

Vocal pitch trainer for Indian classical music with a real-time in-browser tuner

Role: Full-stack and audio/ML engineer

React

FastAPI

NumPy

Web Audio API

AudioWorklet

scikit-learn

GitHub Actions

Built a full-stack singing practice app that records vocals, extracts pitch contours with a custom YIN-style detector, scores them against harmonium and sargam targets in cents, and classifies vocal stability. A real-time tuner runs the same detector on the browser's audio thread.

59x

Faster Pitch Analysis

717 ms to 12 ms at identical accuracy

1.83¢

Mean Pitch Error

100% note detection on a 96-recording benchmark

82

Automated Tests

45 backend + 37 frontend, run in CI

Problem

Singers practicing Hindustani music against a harmonium get no objective feedback. Generic tuners don't understand sargam, thaats, or a movable Sa, and can't tell you which notes you consistently miss.

Solution

SurSadhana AI pairs a playable harmonium with recorded and live pitch analysis. It returns cents-level feedback, classifies vocal stability, scores whole phrases, and tracks which swaras you tend to sing sharp or flat.

Architecture

- React client: harmonium keyboard, practice, sargam, and progress pages
- AudioWorklet live tuner running YIN on the browser audio thread
- Recordings compressed to 16 kHz WAV and uploaded to FastAPI
- FFT-based YIN pitch contour extraction and cents-offset feedback
- Stability classification from signal-processing features
- Viterbi aligner for multi-note phrases; Vercel + Render deployment with CI

Highlights

- Custom YIN-style detector, vectorized in NumPy with an FFT-based difference function (717 ms to 12 ms)
- 100% note detection and 1.83-cent mean error on a 96-recording synthetic benchmark
- Real-time tuner: the detector is ported to an AudioWorklet, so live feedback needs no server round-trip
- Onset-aware phrase scoring with a monotonic Viterbi alignment of sung frames to target notes
- Models all ten Hindustani thaats, with alankar drills, a tanpura drone, and 'Find my Sa' calibration
- Installable PWA with 82 automated tests (45 backend, 37 frontend) in CI

Challenges and Tradeoffs

- Challenge: Pure-Python pitch detection took 717 ms per clip. Change: Vectorized YIN with an FFT difference function, reaching 12 ms with identical accuracy.
- Challenge: Live feedback through the server was too slow. Change: Ported the detector to an AudioWorklet so the tuner runs entirely in the browser.
- Challenge: Unevenly held notes shifted every later scoring window. Change: A monotonic Viterbi alignment maps frames to notes where the pitch actually changes.
- Tradeoff: A heavy ML runtime vs a light server. Decision: NumPy-only inference at runtime, with a heuristic fallback.

Outcomes

- A public, installable app that turns practice sessions into measurable pitch data
- Per-swara analytics tell singers exactly which note to work on
- A benchmark script makes the latency and accuracy numbers reproducible

Live demo: <https://sur-ai.vercel.app> · Repository: <https://github.com/Jaskeerat/sur-ai> · Full case study: <https://jaskeerat-portfolio.vercel.app/projects/sursadhana-ai>